# Research Proposal

Learning How to Think: Hierarchical Computation Allocation for Reinforcement Learning of Long-Horizon LLM Agents

## 1 Motivation

**The amount of computation used for reasoning is externally specified rather than learned by the agent.**

In current reasoning systems, computational resources are typically controlled by manually designed inference parameters, including:

- maximum reasoning token budgets,
- fixed reasoning horizons,
- predefined rollout lengths,
- fixed numbers of sampled reasoning trajectories.

Formally, most existing reasoning pipelines assume a fixed computation horizon:

$$x \to z_1, z_2, \ldots, z_T \to y,$$

where $x$ denotes the input problem, $z_t$ represents intermediate reasoning steps, $y$ is the final output, and the reasoning horizon $T$ is determined before inference.

However, reasoning difficulty is highly variable across tasks. Allocating the same amount of computation to every problem is therefore inefficient. Existing systems suffer from two opposite failure modes:

- **Over-computation:** the agent performs unnecessary reasoning, redundant self-reflection, or excessive exploration, resulting in wasted computational resources.
- **Under-computation:** the agent terminates reasoning too early, fails to explore sufficient reasoning paths, and produces incomplete solutions.

Humans naturally address this problem through adaptive cognitive resource allocation: simple tasks receive limited deliberation, while difficult tasks trigger additional reasoning effort. This suggests that computation itself should be treated as a decision variable rather than a fixed inference parameter.

Therefore, this work investigates the following fundamental question:

**How can an LLM agent autonomously decide how much computation to allocate during reasoning?**

## 2 Research Gap

Existing reinforcement learning methods primarily focus on optimizing:

"What action should the agent take next?"

by improving reasoning trajectories, policy optimization, and credit assignment.

Recent studies have started exploring:

"When should an agent stop reasoning?"

However, stopping represents only one extreme case of a more general computation allocation problem.

The fundamental challenge is:

**How should an intelligent agent allocate computational resources throughout the reasoning process?**

A general computation-aware reasoning agent should learn:

1. whether additional computation is expected to improve future reasoning outcomes;
2. how much computation should be allocated at each reasoning stage;
3. how computation allocation should adapt according to task difficulty and reasoning state.

Despite its importance, computation allocation remains largely treated as a manually designed hyperparameter. Existing approaches specify reasoning budgets externally, while the agent itself has no mechanism to evaluate the marginal value of additional computation.

This creates a fundamental gap:

**Current RL agents learn how to reason, but not how much to reason.**

## 3 Research Question

This work investigates whether an LLM agent can learn computation allocation as a sequential decision-making problem through reinforcement learning.

Specifically, we study whether an agent can jointly learn:

- **what reasoning actions to perform** through a reasoning policy;
- **how much computation to allocate** through a computation allocation policy;
- **whether additional reasoning is worthwhile** through a learned computation value function;
- **when to terminate reasoning** as a special case of allocating zero additional computation.

The central hypothesis of this work is:

**Reasoning computation can be learned as an adaptive resource allocation policy rather than a manually specified budget.**

Rather than requiring predefined reasoning horizons, the proposed framework allows an agent to dynamically determine its computational investment based on the expected utility of additional reasoning.

# 4 Key Idea

We propose **Hierarchical Computation Allocation (HCA)**, a reinforcement learning framework that explicitly models reasoning computation as a sequential decision-making problem.

Instead of treating computation as a fixed inference budget,

$$\text{Question} \rightarrow \text{Reason}^{512} \rightarrow \text{Answer},$$

our framework introduces a hierarchical controller that dynamically allocates computation during reasoning,

$$\text{Question} \rightarrow \text{Computation Planner} \rightarrow \text{Allocate Computation} \rightarrow \text{Reason} \rightarrow \text{Reallocate} \rightarrow \text{Answer}.$$

The proposed framework enables an LLM agent to learn not only *how* to reason, but also *how much* reasoning computation is worth performing for each problem.

# 5 Main Contributions

This work introduces a new reinforcement learning framework that formulates reasoning as a hierarchical computation allocation problem instead of a fixed-horizon sequence generation problem.

## 5.1 Contribution 1: Hierarchical Computation Allocation

We formulate reasoning computation as a hierarchical sequential decision process, where an LLM agent learns to allocate computational resources before generating reasoning trajectories.

The proposed framework consists of two interacting components: a high-level computation planner and a low-level reasoning executor.

At each reasoning state $h_t$, the computation planner determines the amount of reasoning resources to allocate:

$$b_t \sim \pi_\phi(\cdot|h_t),$$

where $b_t$ denotes the allocated computation budget and $\pi_\phi$ represents the computation allocation policy.

The computation planner parameterized by $\phi$ produces a probability distribution over a predefined set of computation budgets:

$$\mathcal{B} = \{b_1, b_2, \ldots, b_K\}.$$

Specifically, the planner first computes allocation logits:

$$s_t = f_\phi(h_t),$$

which are converted into a categorical distribution over possible budgets:

$$\pi_\phi(b|h_t) = \frac{\exp(s_b)}{\sum_{b' \in \mathcal{B}} \exp(s_{b'})}.$$

The computation budget is then sampled as:

$$b_t \sim \pi_\phi(\cdot|h_t).$$

This sampled budget determines the amount of computation available to the reasoning executor.

Conditioned on the allocated budget, the reasoning executor generates the corresponding reasoning segment:

$$z_{t:t+b_t} \sim \pi_\theta(\cdot|h_t, b_t),$$

where $\pi_\theta$ represents the reasoning generation policy.

The two policies form a hierarchical factorization of the joint decision process. By the chain rule of probability, the hierarchical policy is defined as:

$$\pi_{\theta,\phi}(b_t, z_{t:t+b_t}|h_t) = \pi_\phi(b_t|h_t)\pi_\theta(z_{t:t+b_t}|h_t, b_t).$$

This formulation transforms reasoning computation from a fixed inference budget into a learnable allocation decision. However, the computation planner requires a principled mechanism to determine whether allocating additional computation is beneficial. Specifically, the agent must estimate whether the expected improvement from additional reasoning justifies its computational cost. This motivates the introduction of the Computation Allocation Value (CAV).

### 5.2 Contribution 2: Learned Computation Allocation Value

The computation planner introduced in Contribution 1 requires a principled criterion to determine whether allocating additional reasoning computation is beneficial. To address this problem, we introduce the **Computation Allocation Value (CAV)**, which measures the expected utility gain from allocating additional computation while accounting for its cost.

Given the current reasoning state $h_t$, suppose the computation planner considers allocating an additional computation budget $b$. The value of this allocation is defined as:

$$CAV(h_t, b; \lambda) = \mathbb{E}[V(h_{t+b})] - V(h_t) - \lambda b,$$

where:

- $V(h_t)$ denotes the expected task utility of the current reasoning state;
- $\mathbb{E}[V(h_{t+b})] - V(h_t)$ measures the expected improvement obtained by performing additional reasoning;
- $b$ denotes the allocated computation budget (e.g., additional reasoning tokens);
- $\lambda$ represents the learned marginal cost of computation.

The first two terms quantify the benefit of further reasoning, while the last term penalizes excessive computation. Therefore, the optimal computation allocation is obtained by selecting the budget that maximizes the net utility:

$$b_t^* = \arg\max_b CAV(h_t, b; \lambda).$$

If additional reasoning does not provide sufficient benefit to justify its computational cost, then:

$$CAV(h_t, b; \lambda) \leq 0,$$

and the optimal decision becomes:

$$b_t^* = 0,$$

which corresponds to terminating the reasoning process.

Unlike approaches that manually define a fixed computation penalty, we learn $\lambda$ through constrained reinforcement learning. Specifically, the local computation cost term in CAV is derived from a global objective that balances task performance and computation consumption.

For a complete reasoning trajectory $\tau$, the total computation cost is defined as:

$$C(\tau) = \sum_t b_t,$$

where $b_t$ is the computation budget selected by the high-level computation planner at reasoning step $t$.

The computation-aware optimization problem is formulated as:

$$\max_{\theta,\phi} \mathbb{E}[R(\tau)]$$

subject to:

$$\mathbb{E}[C(\tau)] \leq B,$$

where $R(\tau)$ denotes the final task reward, $C(\tau)$ denotes the total reasoning computation consumed by the agent, and $B$ represents the desired average computation budget.

Applying Lagrangian relaxation gives:

$$\mathcal{L}(\theta, \phi, \lambda) = \mathbb{E}\left[R(\tau) - \lambda(C(\tau) - B)\right].$$

Ignoring the constant term $\lambda B$, the optimization objective becomes:

$$J(\theta, \phi) = \mathbb{E}\left[R(\tau) - \lambda C(\tau)\right].$$

Therefore, the computation penalty used in the local CAV formulation,

$$\lambda b_t,$$

is the step-level decomposition of the global computation cost $\lambda C(\tau)$.

The computation price $\lambda$ is updated dynamically during training as a dual variable:

$$\lambda_{k+1} = \lambda_k + \eta_\lambda(\mathbb{E}[C] - B),$$

where $\eta_\lambda$ is the learning rate. If the agent consumes more computation than the desired budget, $\lambda$ increases and makes further computation more expensive. Conversely, if computation usage is below the budget, $\lambda$ decreases and allows the agent to allocate more reasoning resources.

This adaptive update enables the agent to automatically learn the tradeoff between reasoning quality and computational efficiency.

### 5.3 Contribution 3: Hierarchical Reinforcement Learning Optimization

The hierarchical computation allocation framework defines a joint policy that simultaneously optimizes computation allocation and reasoning generation.

At each reasoning step, the computation planner selects a computation budget:

$$b_t \sim \pi_\phi(\cdot|h_t),$$

and the reasoning executor generates the corresponding reasoning segment:

$$z_t \sim \pi_\theta(\cdot|h_t, b_t).$$

Therefore, a complete reasoning trajectory is represented as:

$$\tau = (h_0, b_0, z_0, h_1, b_1, z_1, \ldots, h_T),$$

which contains both the computation allocation decisions and the generated reasoning process. The probability of sampling a trajectory under the hierarchical policy is:

$$\pi_{\theta,\phi}(\tau) = \prod_t \pi_\phi(b_t|h_t)\pi_\theta(z_t|h_t, b_t).$$

**Computation-aware return.** The objective of the agent is not only to maximize task accuracy but also to avoid unnecessary computation. Therefore, we define a computation-aware trajectory return:

$$R'(\tau) = R(\tau) - \lambda C(\tau),$$

where:

- $R(\tau)$ is the original task reward obtained after completing the reasoning trajectory. For example, in mathematical reasoning tasks, $R(\tau)$ can represent whether the final answer is correct.
- $C(\tau)$ is the total computation consumed during reasoning, accumulated from the computation allocation decisions:

$$C(\tau) = \sum_t b_t.$$

- $\lambda$ is the learned computation price that determines the tradeoff between task performance and computational cost.

The term:

$$\lambda C(\tau) = \lambda \sum_t b_t$$

is the trajectory-level computation cost. It is the accumulated version of the local computation cost:

$$\lambda b_t$$

used in the Computation Allocation Value:

$$CAV(h_t, b; \lambda) = \mathbb{E}[V(h_{t+b})] - V(h_t) - \lambda b.$$

Therefore, the CAV provides the local decision criterion, while the computation-aware return provides the global training signal.

**Policy optimization.** The hierarchical policy is optimized by maximizing the expected computation-aware return:

$$J(\theta, \phi) = \mathbb{E}_{\tau \sim \pi_{\theta,\phi}}[R'(\tau)].$$

Expanding the expectation over trajectories:

$$J(\theta, \phi) = \sum_\tau \pi_{\theta,\phi}(\tau) R'(\tau).$$

Taking the gradient with respect to the policy parameters gives:

$$\nabla_{\theta,\phi} J(\theta, \phi) = \nabla_{\theta,\phi} \sum_\tau \pi_{\theta,\phi}(\tau) R'(\tau).$$

Using the log-derivative trick:

$$\nabla_{\theta,\phi} \pi_{\theta,\phi}(\tau) = \pi_{\theta,\phi}(\tau) \nabla_{\theta,\phi} \log \pi_{\theta,\phi}(\tau),$$

we obtain:

$$\nabla_{\theta,\phi} J(\theta, \phi) = \sum_\tau \pi_{\theta,\phi}(\tau) \nabla_{\theta,\phi} \log \pi_{\theta,\phi}(\tau) R'(\tau).$$

Therefore:

$$\boxed{\nabla_{\theta,\phi} J(\theta, \phi) = \mathbb{E}_{\tau \sim \pi_{\theta,\phi}} \left[ \nabla_{\theta,\phi} \log \pi_{\theta,\phi}(\tau) R'(\tau) \right]}$$

In practice, the raw return $R'(\tau)$ is replaced by an advantage estimate to reduce variance:

$$A(\tau) = R'(\tau) - V(h_t),$$

leading to the practical policy gradient:

$$\nabla_{\theta,\phi} J(\theta, \phi) = \mathbb{E}_{\tau \sim \pi_{\theta,\phi}} \left[ \nabla_{\theta,\phi} \log \pi_{\theta,\phi}(\tau) A(\tau) \right].$$

The gradient update jointly improves the two components of the hierarchy:

$$\pi_\phi(b_t|h_t)$$

learns how much computation should be allocated, while:

$$\pi_\theta(z_t|h_t, b_t)$$

learns how to reason effectively under the allocated computation budget.

Meanwhile, the computation price $\lambda$ is updated as a dual variable:

$$\lambda_{k+1} = \lambda_k + \eta_\lambda(\mathbb{E}[C] - B),$$

which automatically adjusts the tradeoff between reasoning accuracy and computational efficiency.

# 6 Method Overview

The proposed framework, **Hierarchical Computation Allocation (HCA)**, formulates reasoning as a sequential decision problem where an LLM agent jointly learns *what to reason* and *how much computation to allocate*.

The overall framework consists of three interconnected components:

1. **Hierarchical Computation Allocation:** A high-level computation planner $\pi_\phi$ determines the reasoning budget $b_t$ based on the current reasoning state $h_t$.
2. **Computation Allocation Value:** The Computation Allocation Value (CAV) evaluates whether allocating additional computation provides sufficient future utility to justify its cost.
3. **Computation-Aware Reinforcement Learning:** The computation planner and reasoning executor are jointly optimized using a reward that balances task performance and computation cost.

## 6.1 Training Pipeline

The training process consists of four stages:

1. **Computation Allocation**

   At reasoning state $h_t$, the computation planner samples a budget:

   $$b_t \sim \pi_\phi(\cdot|h_t)$$

   where $b_t$ represents the allocated reasoning computation.

2. **Reasoning Generation**

   Conditioned on the allocated budget, the reasoning executor generates:

   $$z_t \sim \pi_\theta(\cdot|h_t, b_t)$$

   The process continues until the computation planner selects:

$$b_t = 0,$$

or the maximum reasoning budget is reached.

3. **Computation-Aware Reward Construction**

   After completing a trajectory:

$$\tau = (h_0, b_0, z_0, ..., h_T),$$

   the agent receives:

$$R'(\tau) = R(\tau) - \lambda C(\tau),$$

   where:

$$C(\tau) = \sum_t b_t.$$

4. **Policy and Computation Price Update**

   The hierarchical policy:

$$\pi_{\theta,\phi}(\tau) = \prod_t \pi_\phi(b_t|h_t)\pi_\theta(z_t|h_t, b_t)$$

   is optimized using policy gradients:

$$\nabla_{\theta,\phi} J = \mathbb{E}[\nabla \log \pi_{\theta,\phi}(\tau) A(\tau)].$$

   The computation price is updated by:

$$\lambda_{k+1} = \lambda_k + \eta_\lambda(\mathbb{E}[C] - B).$$

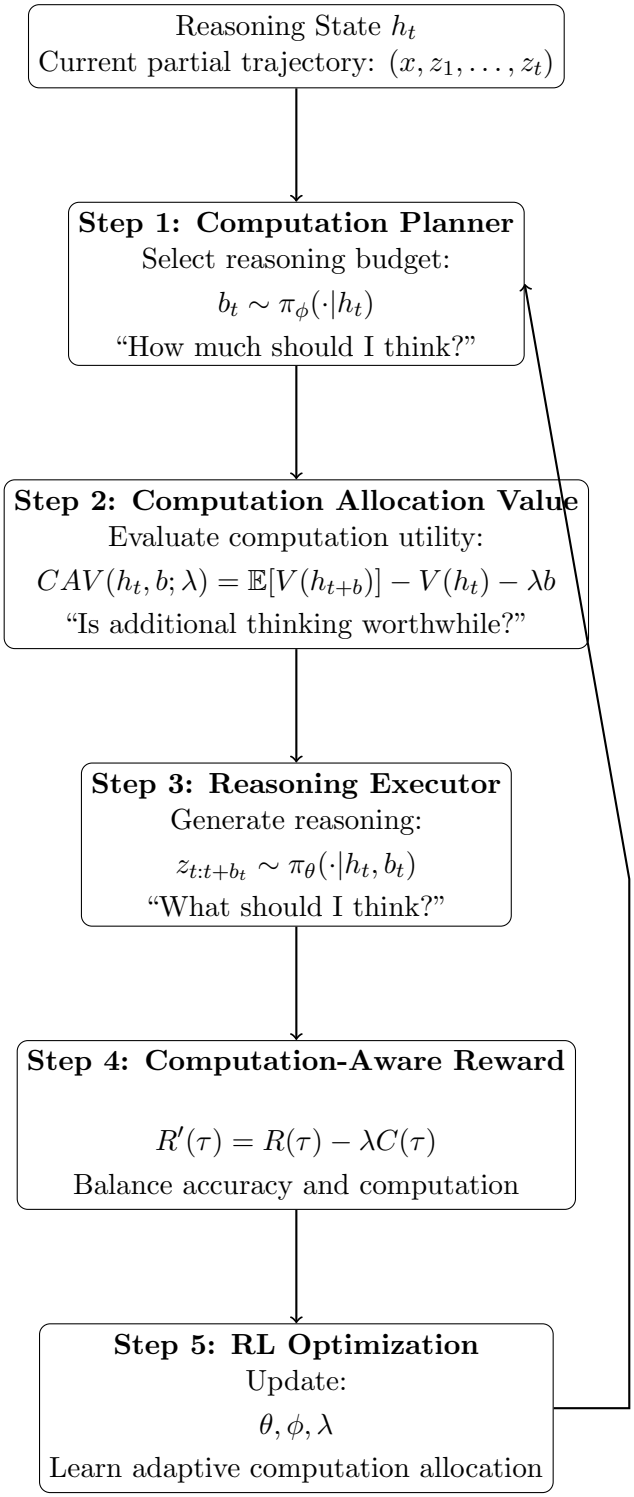


Figure 1: Overview of the proposed Hierarchical Computation Allocation framework. The computation planner selects reasoning budgets, the Computation Allocation Value evaluates their utility, and reinforcement learning optimizes the computation planner, reasoning executor, and computation price.